

ポートフォリオ説明資料

3D ゲームにおけるカメラ制御及び画面描画の基礎研究
Basic Research of Movement of Camera and Rendering in
3D Game

2026

目次

0. 抄録	1
1. はじめに	1
2. データと方法	1
3. 解説	2
3.0. 試作: test ディレクトリ	2
3.1. カメラ制御	2
3.2. 透視投影	3
3.3. 空間上の物体	4
3.4. 設定の保存	4
4. 考察	5
4.1. マジックナンバー	5
4.2. 当たり判定とポリゴン数	5
4.3. 物体の生成	5
4.4. 反省点 1: ヘッダーファイルの煩雑化	6
4.5. 反省点 2: 作業の順序	6
5. まとめ	7
6. 謝辞	7
7. 参考	7

0. 抄録

3D のゲームについて、プレイヤー及びカメラをクォータニオンによって回転させ、空間上の物体を座標変換と透視投影により描画した。移動と回転の速度は個別の設定が可能で、負の値にすると方向が反転する。設定した値は外部のファイルに書き込むことで、ゲーム終了後も保持される機構となっている。また空間上の物体はランダム生成となっており、描画範囲から大きく逸脱した際に削除され、反対側かつ描画範囲の若干外側に再生成される。制作期間は試作が約一ヶ月、移植が二週間となっている。

回転は WASD + QE キーで制御しており、設定の変更やゲームの終了はエスケープキーから行う。またタイトルと設定の画面については十字キーで選択、スペースまたはエンターキーで決定となっている。

作品は GitHub にアップロードした。(<https://github.com/TCAPG22503004/Portfolio>)

1. はじめに

近年のゲーム制作では、Unity や Unreal Engine を始めとするゲームエンジンが用いられることが一般的である。上記二つに関しては、プロジェクトの作成時に 2D または 3D を選択することで大枠が完成している状態から開始することが可能となっている。これにより、ある程度の使用方法を押さえている人は、短時間であっても容易にゲームを形にすることが可能である。これらゲームエンジンは、構想を持つ人がゲームを作る際のハードルを大幅に下げること成功しており、大変素晴らしい技術であると言える。

ところで今年の 12 月中旬に、ふとゲームエンジンが肩代わりしている大枠そのものの仕組みに興味を湧いた。本資料は実際にこれを試した結果をまとめ、反省点を考察することを目的とする。

2. データと方法

コンパイラとして mingw-w64 の x86_64-12.2.0-release-win32-seh-ucrt-rt_v10-rev2.7z を、また対応するバージョンの DX ライブラリ (山田, 2001) をダウンロードした。コンパイル時の bat ファイルは、Nkmrtkhd (2020) の記事を参考に作成した。ファイルの操作は Windows Subsystem for Linux のバージョン 2.6.3.0 を用いた。OS は Ubuntu 22.04.5 LTS(Jammy Jellyfish)である。テキストエディタは Vim を使用した。データの公開とバックアップを兼ね、作品は GitHub (<https://github.com/TCAPG22503004/Portfolio>) にアップロードした。

概要としては、カメラすなわちプレイヤーの回転量を表すクォータニオンを保持し、回転した座標系における直方体の座標を用い、透視投影によってスクリーンに描画するという流れとなっている。3章ではそれぞれの工程について、コードを含めて解説する。

3. 解説

3.0. 試作: test ディレクトリ

始めに Python 及び Pygame を用いて研究と試作を行った。初期では回転行列を使用していた (Matrix*.py) が、調べた結果と先生からのアドバイスに基づきクォータニオンに切り替えている。また動作研究の過程で、一人称視点と三人称視点は二行の書き換え、あるいは if 文を五行程度追加することで切り替えられることが分かった (Camera1st.py, Camera3rdPerson.py)。完成系が確認できた (Final.py) 後に、C++での制作を開始した。

3.1 カメラ制御

カメラはクォータニオンにより回転させており、quaternion.cpp の関数で計算している。計算式は矢田部 (2007) の記事から引用している。すなわち、クォータニオン p とクォータニオン q の積は

$$\tilde{q}\tilde{p} = \begin{pmatrix} q_0 & -q_1 & -q_2 & -q_3 \\ q_1 & q_0 & -q_3 & q_2 \\ q_2 & q_3 & q_0 & -q_1 \\ q_3 & -q_2 & q_1 & q_0 \end{pmatrix} \begin{pmatrix} p_0 \\ p_1 \\ p_2 \\ p_3 \end{pmatrix}$$

位置ベクトル r の回転は

$$r' = \begin{pmatrix} q_0^2 + q_1^2 - q_2^2 - q_3^2 & 2(q_1q_2 - q_0q_3) & 2(q_1q_3 + q_0q_2) \\ 2(q_1q_2 + q_0q_3) & q_0^2 - q_1^2 + q_2^2 - q_3^2 & 2(q_2q_3 - q_0q_1) \\ 2(q_1q_3 - q_0q_2) & 2(q_2q_3 + q_0q_1) & q_0^2 - q_1^2 - q_2^2 + q_3^2 \end{pmatrix} r$$

座標系 r の回転は

$$r' = \begin{pmatrix} q_0^2 + q_1^2 - q_2^2 - q_3^2 & 2(q_1q_2 + q_0q_3) & 2(q_1q_3 - q_0q_2) \\ 2(q_1q_2 - q_0q_3) & q_0^2 - q_1^2 + q_2^2 - q_3^2 & 2(q_2q_3 + q_0q_1) \\ 2(q_1q_3 + q_0q_2) & 2(q_2q_3 - q_0q_1) & q_0^2 - q_1^2 - q_2^2 + q_3^2 \end{pmatrix} r$$

とした。

3.0 章で述べた試作は $(x, y, z) = (\text{右}, \text{上}, \text{奥})$ 向き正の左手系であるが、 z 座標を高さ方向に取る方が一般的であると考え、本制作では $(x, y, z) = (\text{右}, \text{奥}, \text{上})$ 向きを正とする右

手系を採用している。カメラの回転はワールド座標に基づいており (void ProductWorld)、ローリングが可能である。対して後述する物体の生成ではローカル座標で回転させているため、積の順番を逆にした関数を別途用意した(void ProductLocal)。

ゲーム内では、(SW, EQ, AD) キーを入力することで (x, y, z) 軸を中心に回転が可能である。具体的には、0 番目が $\cos \theta$ で、1 から 3 番目は対応する成分が $\sin \theta$ 、他の二成分は 0 となるクォータニオンと、現在のクォータニオンとの積を取ることで回転させている。またカメラの移動は、速度 v について $(0, v, 0)$ で表される 3 次元ベクトルを、クォータニオンで回転させることで表現している。キー入力と移動の処理は player.cpp に記載されている。

3.2. 透視投影

空間上の物体について、3.1 章の式で回転させた座標を用い、透視投影でスクリーン上の座標を求めた。三谷 (2015) の資料を参考に、perspective.cpp に記載している。

Remi (2014) は透視図法の画角による歪みについて、45 度程度であれば問題が無いことを報告している。そのため本作品においても、左右の視野角は $\pi/4$ とした。スクリーンの大きさを sx, sy とすると、スクリーンとカメラ間の距離 d 、ないし空間上の座標 (x, y, z) をスクリーンに投影した座標 (X, Y) は、相似比より

$$d = \frac{sx}{\tan\left(\frac{\pi}{4} \cdot \frac{1}{2}\right)}$$

$$X = d \frac{x}{y} + sx$$

$$Y = -d \frac{z}{y} + sy$$

と表される(図 1)。ただし、空間上の座標は $(x, z) =$ (右, 上) 向きが正、スクリーン座標は左上原点の $(x, y) =$ (右, 下) 向きが正であることに注意が必要である。

蛇足であるが、三人称視点はプレイヤーがスクリーンまで移動した状態であると解釈でき、 X, Y 共に分母を $(y + d)$ と置き換えることで計算が可能である。

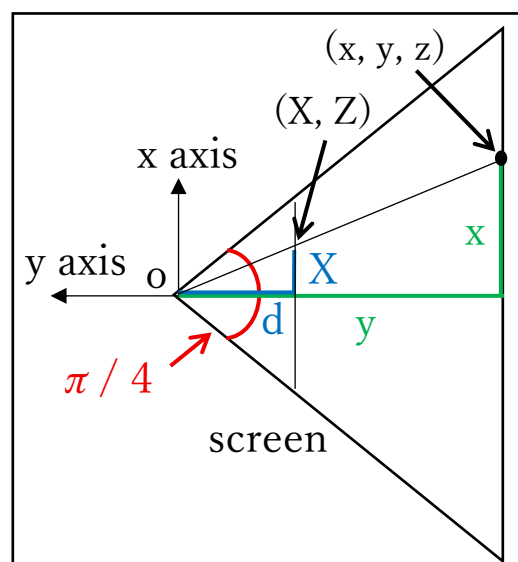


図 1. 透視投影の模式図。
三谷(2015) を参考に作成。

3.3. 空間上の物体

多くのゲームにおいて、マップは事前に作成されたものが用意されている。しかしながら今回は時間と性能の都合上、直方体をランダムに 10 個配置する仕様とした。

直方体は、15 の直線を順に結ぶことで表現した (図 2)。線が重複する分余計な処理が増えているが、3.0 章の試作の際にこちらの方が都合が良く、そのまま受け継いだ形となっている。

ゲーム開始時の配置は乱数で無作為に決定しており、以降は見切れた方向の反対側に新しく作成している (図 3)。例えば図形の相対位置が余白部分のさらに右側 (図 3, 赤) となった場合に削除し、左側の余白内 (図 3, 青) に新しく生成している。この際、z 座標と図形の大きさはランダムな値となっている。

生成の方向は $(0, \text{distance}, 0)$ で表される三次元ベクトルを、プレイヤーのクォータニオンを用いて回転させることで得た。ただし、実際に回転しているプレイヤーに対し、物体は相対的な見かけ上の回転であるため、クォータニオンの角度を負の値とする必要がある。 $\cos(-\theta) = \cos \theta$, $\sin(-\theta) = -\sin \theta$ であるため、使用したクォータニオンは虚部、すなわち 1 から 3 番目の要素に -1 を掛けたものである。

以上の処理は、object.cpp と game.cpp で行われている。

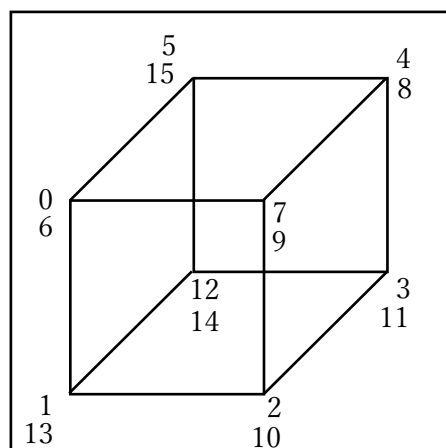


図 2. 直方体の描画順。

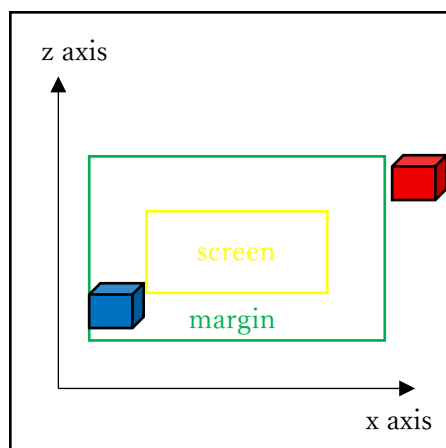


図 3. 生成位置の模式図。

赤は削除、青は生成位置を示す。

3.4. 設定の保存

カメラ操作の仕様として、速度と反転の双方を考慮する必要がある。そこで速度の値をゲーム内で設定し、外部のファイルに書き込むことでゲーム終了後も値が保持されるように設計した。

まず、設定画面を開いた際に、ファイルからそれぞれの値を読み込み表示している。設定画面から離れる際にファイルに書き込み、ゲーム開始時と設定画面からゲームに戻る際にファイルの値を変数に代入している。以上の処理は config.cpp で完結させる予定であったが、ファイルの書き込みが出来なかったため、急遽 write.cpp を後付けで作成した。不

具合の原因は不明であるが、解決でき次第 config.cpp に統一し、write.cpp は削除する予定であるため、write のヘッダーファイルは作成していない。

4. 解説

4.1. マジックナンバー

本作品では、座標とクォータニオンの計算でマジックナンバーを使用している。座標は3成分、クォータニオンは4成分で表現され、恐らく変更されることは無いと考えた。

3.3 節で、15 の直線から図形を生成していることを述べた。この数値は配列の宣言で使用するため、constant.hpp から define を用いて定義している。ここに上記二つの数を記載しても良いように思えるが、define により配列の大きさを外部で決定できることを完成後に知り、置換の見落としが発生する恐れがあったため手を加えていない。次回以降の、新規でプロジェクトを作成する際には積極的に採用したいと考える。

4.2. 当たり判定とポリゴン数

アクションゲームにおいて必須となる機能が当たり判定である。一般に、円では半径、座標系に直交する四角形ではその座標、線分では外積等が用いられているように感じる。

今作品はプレイヤーが大きさの無い点であり、対象の図形は直方体、かつ全ての辺が x, y, z 軸のいずれかを向いている。そのため、それぞれの最小及び最大値との比較により、一つの図形あたり6回の計算で当たりを取ることが可能である。今回はあくまで基礎研究であるため体力やスコア等は省略したが、ゲームに落とし込む難易度自体はそこまでであると考えられる。

しかしながら先にも少し触れたように、昨今のゲームでは背景や小物すらも滑らかに表現されている。今回のように全ての頂点で当たりの計算をすることは、とても現実的ではないと断言できる。そも当たり判定以前に、描画自体の処理が重くなり過ぎることが懸念される。少なくとも、裏側と呼べる部分の処理をカットする機構が必要不可欠である。

4.3. 物体の生成

3.3 節で述べた通り、本作品では直方体が画面外のランダムな位置に生成される。そのため一切の回転をせずに放置した際に、画面内に一つも描画されない状況が生じる。

この対策として、新たな物体を奥の方向に生成する場合も試した。しかしながら何も無い空間に突如として出現する様子が、まるで処理落ちしているような感覚を想起させたため、没となった。地球のように球面状に座標を取り、地平線を定義することで自然な表示が出来るのではないかと考えられる。ただしこれは水平方向の話であり、空に向かって真っすぐ上昇している場合、カメラを向けた時点で表示されていなければならず、別の手法を採る必要がある。そしてこれも先に述べたが、実際のゲームでは事前にマップが定義されていることが多く、厳密さを求める際には全ての雲や星の位置を決定しなければならない。以上から、ここから自然さを求める場合は、4.2 節でも述べたような最適化が必要である。

4.4. 反省点 1: ヘッダーファイルの煩雑化

今回の作品では、複数の関数にまたがって使用されている変数はグローバルに定義している。これが災いして、ヘッダーファイルの変数の数がかなり多く、視覚的に望ましくない形となっている。

解決策として、ループの度に値が変動する変数はループの中で定義し、他の関数の引数として渡す方法が挙げられる。現時点での関数の引数はほとんどが 0 であり、多少増やしても見栄えの悪化はあまり深刻化しないことが予測される。

加えて、初期化をヘッダーファイルで行わない手段もある。一部のクラスでは、DX ライブラリの関数を使用するため、Init 関数を定義している。これを全てのクラスに適用し、パラメータは cpp ファイルで調整するというルールに統一する手段も有効であると考えられる。

4.5. 反省点 2: 作業の順序

3.0 章で試用品を作成したこともあり、C++ の本制作では各部品を独立して作成し、後から組み合わせる手法を採った。しかしながら、全体の進捗が分かりにくく、モチベーションへの悪影響があった。加えて、全てを組み合わせるまでないと関数の動きを見ることが出来ず、不具合発生の由来を特定する時間が増加したという問題もあった。試作の時と同様に、仕様を一つずつ順に確認しながら制作を進めた方が、より効率良く進めることが出来たのではないかと反省している。

5. まとめ

何となくの思い付きから、3D ゲームの基礎部分を研究した。クォータニオンや透視投影、及び関連する複数の概念については知らない分野であったため、まずこれらの理解に勤しんだ。その後に得られた知見を基に、C++で動くモノを創り上げた。複数の課題が浮き彫りとなったため、今後の制作に活かしたい所存である。

6. 謝辞

当ファイルは GitHub にアップロードする都合上、本名は伏せさせていただきます。

本研究を進めるにあたり、回転系や数学等の専門的な相談に快く乗って下さった Y 先生に深く感謝申し上げます。また担任の T 先生には、普段の学校生活を陰ながらサポートいただいております。記して感謝申し上げます。最後に、常日頃からお世話になっている全ての方々へ感謝の意を表します。

7. 参考

- 三谷 純 (2015). コンピュータグラフィックス基礎 第3回 3次元の座標変換. https://mitani.cs.tsukuba.ac.jp/lecture/old2015/cg/03/03_slides.pdf , (参照 2026-02-13).
- Nkmrtkhd (2020). Windows のバッチのループ内で変数を置換する. *Qiita*. <https://qiita.com/nkmrtkhd/items/100949962a3b216fe524> , (参照 2026-02-13).
- Remi (2014). 歪みの原因 - 画角. パースフリークス. <http://www.persfreaks.jp/main/aov/dist2/> , (参照 2026-02-13).
- 山田 巧 (2001). DX ライブラリ置き場. <https://dxlib.xsrv.jp/> , (参照 2026-02-12).
- 矢田部 学 (2007). クォータニオン計算便利ノート. *MSS 技報・Vol. 18*. https://www.mesw.co.jp/business/report/pdf/mss_18_07.pdf , (参照 2026-02-13).